Структура работы

Лабораторная работа №1 — регрессия нейронными сетями на табличных данных, NASA PCoE Battery Dataset (34 батареи)

Документ собран автоматически 2026-10-03 · в проекте 154 файлов, 204.2 МБ без папки _trash · сборщик: scripts/build_structure_doc.py

---

1. Как пользоваться документом

Документ отвечает на вопрос «что где лежит»: по папкам, по файлам и по командам, которые их создают. Он не пересказывает результаты исследования — за ними идёт отчёт (reports/lab1_report.md).

- разделы 2–3 — общая логика и схема папок: достаточно, чтобы сориентироваться;
- разделы 4–11 — что лежит в каждой папке и какие файлы главные;
- раздел 12 — команды воспроизведения;
- раздел 13 — правила: куда класть новый файл и что можно удалить;
- приложение A — полный перечень файлов с размерами.

> Все числа в документе (счётчики, размеры, списки) читаются с диска при сборке, поэтому документ описывает текущее состояние папок, а не состояние на момент написания.

2. Логика проекта за тридцать секунд

Проект разложен на пять слоёв. Каждый слой отвечает за своё и не лезет в чужую территорию: данные не содержат кода, документы не содержат результатов экспериментов, а числа в отчёте берутся только из data/processed/.

Слой	Папки	Что внутри	Правило
Данные	data/raw, data/processed	исходные файлы NASA (.mat) и результаты: таблицы, метрики, веса моделей	data/raw не изменяем никогда; data/processed целиком перезаписывается пайплайном
Код	src	15 файлов: 14 модулей разбора, чистки, признаков, RUL, сплита, обучения, графиков и отчёта	единственный источник чисел для отчёта
Точки входа	scripts	12 скриптов, которые запускаются руками	всё, что запускается человеком, живёт здесь, а не в src
Проверки	tests, scripts/check_leakage.py, scripts/check_parser.py	24 автотеста, 4 проверки честности обучения и 7 проверок разобранных данных	зелёный pytest и check_leakage — обязательное условие сборки отчёта
Документы	reports, docs	отчёт, разбор работы, версия для защиты, схема процесса, настоящий документ	пишутся для человека, собираются скриптами из md/JSON

3. Схема папок

```
battery_project/
├── artifacts/
│   ├── battery_stats.csv
│   └── no_eol_batteries.json
├── data/
│   ├── interim/  · 0 файлов, 0 Б
│   ├── processed/  · 34 файла, 9.1 МБ
│   └── raw/  · 34 файла, 189.3 МБ
├── docs/
│   ├── lab1_defense.html
│   ├── lab1_defense.md
│   ├── lab1_likbez.html
│   ├── lab1_likbez.md
│   ├── lab1_process.css
│   ├── lab1_process.html
│   ├── lab1_process.xml
│   ├── lab1_process.xsd
│   ├── lab1_process.xsl
│   ├── project_structure.docx
│   ├── project_structure.md
│   ├── project_structure.pdf
│   └── style_likbez.css
├── reports/
│   ├── figures/  · 33 файла, 5.0 МБ
│   ├── lab1_report.html
│   ├── lab1_report.md
│   ├── style.css
│   └── theory_manual.md
├── scripts/
│   ├── build_doc_html.py
│   ├── build_html_report.py
│   ├── build_structure_doc.py
│   ├── check_leakage.py
│   ├── check_parser.py
│   ├── diagnose_batteries.py
│   ├── regen_figures.py
│   ├── run_all.py
│   ├── run_baseline_chart.py
│   ├── run_baselines.py
│   └── xml_to_html.py
├── src/
│   ├── __init__.py
│   ├── baselines.py
│   ├── clean.py
│   ├── config.py
│   ├── export.py
│   ├── features.py
│   ├── html_report.py
│   ├── infographics.py
│   ├── main.py
│   ├── model.py
│   ├── parser.py
│   ├── plots.py
│   ├── rul.py
│   ├── split.py
│   └── validate.py
├── tests/
│   ├── test_features.py
│   ├── test_parser.py
│   ├── test_rul.py
│   └── test_split.py
├── .gitignore
├── README.md
└── requirements.txt
```

> В дереве не показан 1 файл: он не относится к пайплайну и документам и посчитан в приложении Б.

Папка	Файлов	Размер	Роль в работе
data/	68	198.5 МБ	сырые данные NASA и всё, что из них получено
src/	15	147 КБ	библиотека кода пайплайна
scripts/	12	97 КБ	запускаемые скрипты
tests/	4	8 КБ	автотесты pytest
reports/	37	5.2 МБ	отчёт и 33 графика
docs/	13	307 КБ	разбор, версия для защиты, схема процесса
artifacts/	2	2 КБ	промежуточные сводки по батареям

4. Корень проекта

Файл	Размер	Назначение
README.md	9 КБ	точка входа: метрики, честное сравнение с базовыми моделями, проверки, ограничения, список документов
requirements.txt	147 Б	11 пакетов с минимальными версиями: numpy, pandas, scipy, scikit-learn, tensorflow, keras, matplotlib, pyarrow, openpyxl, joblib, pytest
.gitignore	438 Б	что не коммитится: data/raw, data/processed, reports/figures, *.keras, кэши Python и IDE

> Папка data/interim/ существует, но пустая: промежуточные таблицы пишутся сразу в data/processed/. Она указана в .gitignore и в src/config.py (INTERIM_DIR), а check_parser.py и diagnose_batteries.py используют её как запасной путь поиска raw_cycles.parquet.

5. data/ — данные

data/raw/ — 34 файла .mat, 189.3 МБ. Это исходный датасет NASA PCoE: по одному файлу на батарею (B0005…B0056, 34 батареи). Каждый файл — структура MATLAB с массивом циклов трёх типов: charge, discharge, impedance; внутри цикла лежат измерения напряжения, тока, температуры, времени и ёмкости. Папка read-only: парсер только читает её, размер файлов не меняется.

data/processed/ — 34 файла, 9.1 МБ. Здесь лежит всё, что порождено пайплайном, и это единственный источник чисел для отчёта.

Группа файлов	Файлы	Зачем
Разобранные циклы	raw_cycles.parquet, clean_cycles.parquet	одна строка на разрядный цикл: до и после минимальной чистки
Таблица признаков	battery_features.parquet / .csv / .json / .xlsx, battery_features_report.md	признаки и цели для всех 34 батарей; xlsx и json — для просмотра и внешних инструментов
Датасет задачи A	battery_rul.parquet (120 КБ)	остаточный ресурс, 8 батарей с полной кривой деградации
Датасет задачи B	battery_voltage.parquet (369 КБ)	среднее напряжение следующего цикла, 34 батареи
Датасет задачи C	battery_capacity.parquet (289 КБ)	ёмкость следующего цикла, 33 батареи (одна отсеяна как слишком короткая)
Метрики моделей	metrics_rul.json, metrics_voltage.json, metrics_capacity.json, summary_metrics.json	MAE / RMSE / R^2 отдельно для train, val, test и сводка по трём задачам
Метрики базовых	metrics_baselines_{rul,voltage,capacity}.json, summary_baselines.json, comparison_baselines.json	константа, линейная регрессия и Random Forest на том же разбиении; summary_baselines.json — источник раздела 5.4 отчёта
Обученные модели	model_rul.keras, model_voltage.keras, model_capacity.keras	веса MLP по трём задачам; нужны для перерисовки графиков и пересчёта предсказаний
Диагностика	battery_diagnostics.csv	сводка по батареям: число циклов, ёмкость в начале и в конце, оценка отказа

6. src/ — код пайплайна

15 файлов, 147 КБ. Модули вызываются по порядку оркестратором src/main.py; каждый отвечает за один этап и пишет результат в data/processed/.

Модуль	Что делает
config.py	пути, константы (пороги ёмкости, число слоёв, seed, patience), списки батарей и признаки трёх задач
parser.py	чтение .mat → одна строка на разрядный цикл: статистики измерений, длительность предыдущего заряда, значения импеданса
clean.py	минимальная чистка: удаление физически невозможных значений, MAD-фильтр импеданса, заполнение вперёд запрещено
features.py	таблица признаков: измерения цикла, лаги, разности, скорость разряда, нормированная ёмкость, цели следующего цикла
rul.py	остаточный ресурс в циклах и в долях для 8 батарей с полной кривой
split.py	разбиение строго по батареям (GroupShuffleSplit), проверка отсутствия пересечений
validate.py	проверки после каждого этапа: состав батарей, корректность целей, отсутствие пересечений в разбиении
model.py	одна архитектура MLP на три задачи: обучение, ранняя остановка, сохранение весов и метрик
baselines.py	три базовых предсказателя: константа, линейная регрессия, Random Forest — на том же разбиении
plots.py	графики задач: история обучения, предсказания, остатки, кривые деградации
infographics.py	дополнительные графики: важность признаков, корреляции, аномалии, сводная панель
export.py	выгрузка таблиц (csv / xlsx / json) и генерация reports/lab1_report.md
html_report.py	сборка HTML-отчёта из markdown через pandoc, стили reports/style.css
main.py	оркестратор: восемь этапов от .mat до отчёта

7. scripts/ — точки входа

12 файлов, 97 КБ. Скрипты запускаются вручную, каждый делает одну вещь и печатает, что получилось.

Скрипт	Команда	Что делает
run_all.py	py -X utf8 scripts/run_all.py	весь пайплайн целиком: разбор → чистка → признаки → обучение → графики → отчёт
run_baselines.py	py -X utf8 scripts/run_baselines.py	обучает три базовых предсказателя на том же разбиении, пишет metrics_baselines_*.json, summary_baselines.json и три графика
run_baseline_chart.py	py -X utf8 scripts/run_baseline_chart.py	строит figures/baseline_comparison.png из готового summary_baselines.json, без обучения
regen_figures.py	py -X utf8 scripts/regen_figures.py	перерисовывает три графика отчёта из сохранённых таблиц и весов, без обучения
build_html_report.py	py -X utf8 scripts/build_html_report.py	собирает reports/lab1_report.html из markdown
build_doc_html.py	py -X utf8 scripts/build_doc_html.py	собирает docs/lab1_likbez.html и docs/lab1_defense.html
xml_to_html.py	py -X utf8 scripts/xml_to_html.py	валидирует схему процесса по XSD, применяет XSLT, пишет docs/lab1_process.html и проверяет результат
build_structure_doc.py	py -X utf8 scripts/build_structure_doc.py	собирает настоящий документ: md, docx, pdf
check_leakage.py	py -X utf8 scripts/check_leakage.py	четыре проверки честности: состав признаков, направление лагов, разбиение, обучение препроцессинга только на train
check_parser.py	py -X utf8 scripts/check_parser.py	семь проверок разобранных данных: число циклов, убывание ёмкости, температуры, пропуски
diagnose_batteries.py	py -X utf8 scripts/diagnose_batteries.py	три способа посчитать порог отказа и отбор батарей с длинной кривой; пишет battery_diagnostics.csv

8. tests/ — автотесты

4 файла, 8 КБ, 24 теста, ~5 секунд. Запуск: py -X utf8 -m pytest tests -q. Тесты работают на двух реальных батареях (B0005, B0006) из data/raw и на синтетическом списке батарей для разбиения.

Файл	Что проверяет
conftest.py	общие настройки pytest и пути к проекту
test_parser.py	разбор .mat: непустой результат, наличие колонок, последовательные номера циклов, все 34 батареи
test_features.py	наличие лагов и производных, корректность нормировки, цели следующего цикла
test_rul.py	остаточный ресурс: неотрицателен, убывает, последний цикл = 0, доля в диапазоне 0…1
test_split.py	разбиение: нет пересечений, все батареи покрыты, размеры частей известны, повторный запуск даёт тот же результат

9. reports/ — отчёт

Файл	Размер	Назначение
lab1_report.md	48 КБ	исходник отчёта: 11 разделов, 18 графиков, все числа из data/processed/
lab1_report.html	57 КБ	собранная HTML-версия с оглавлением и локальными стилями
style.css	3 КБ	стили отчёта (локальный файл, внешних подключений нет)
theory_manual.md	24 КБ	справочник по теории: MLP, метрики, разделение, интерпретация результатов
figures/	33 PNG, 5.0 МБ	18 графиков подключены к отчёту

В figures/ лежит 33 PNG, из них 18 подключены к отчёту (перечень — в отчёте, раздел «Приложение»). Остальные 15 файла — вспомогательные графики: часть из них строится скриптом run_baselines.py, часть осталась от промежуточных запусков и на отчёт не влияет.

10. docs/ — документы

Файл	Размер	Назначение
lab1_likbez.md / .html	41 КБ	подробный разбор работы: предмет, данные, пайплайн, модели, базовые предсказатели, результаты, открытые вопросы
lab1_defense.md / .html	19 КБ	короткая версия для устной защиты: рассказ, ключевые числа, словарик терминов, вероятные вопросы
lab1_process.xml	52 КБ	схема процесса в XML: этапы, сущности, параметры, команды
lab1_process.xsd	16 КБ	схема валидации XML (проверяется при каждой сборке)
lab1_process.xsl	20 КБ	преобразование XML в HTML
lab1_process.css	4 КБ	стили страницы схемы процесса
lab1_process.html	72 КБ	собранная страница схемы процесса
style_likbez.css	6 КБ	стили документов likbez и defense
project_structure.md / .docx / .pdf	—	настоящий документ: структура папок, файлы, команды, правила

11. artifacts/ — промежуточные сводки

2 файла, 2 КБ. Папка не участвует в пайплайне: это справки по батареям, собранные при разборе данных.

Файл	Размер	Содержимое
battery_stats.csv	2 КБ	сводная статистика по батареям (число циклов, диапазон ёмкости)
no_eol_batteries.json	927 Б	список батарей, у которых порог отказа не достигнут

12. Команды: что чем пересобирается

Задача	Команда	Что изменится на диске
Полный пересчёт	py -X utf8 scripts/run_all.py	data/processed/*, reports/figures/*, reports/lab1_report.md
Тесты	py -X utf8 -m pytest tests -q	ничего
Проверка честности	py -X utf8 scripts/check_leakage.py	ничего
Базовые предсказатели	py -X utf8 scripts/run_baselines.py	metrics_baselines_*.json, summary_baselines.json, comparison_baselines.json, три графика
График сравнения	py -X utf8 scripts/run_baseline_chart.py	figures/baseline_comparison.png
Перерисовка графиков	py -X utf8 scripts/regen_figures.py	capacity_degradation.png, 07_anomalous_batteries.png, 09_rul_vs_rul_frac.png
HTML отчёта	py -X utf8 scripts/build_html_report.py	reports/lab1_report.html
HTML документов	py -X utf8 scripts/build_doc_html.py	docs/lab1_likbez.html, docs/lab1_defense.html
Схема процесса	py -X utf8 scripts/xml_to_html.py	docs/lab1_process.html
Настоящий документ	py -X utf8 scripts/build_structure_doc.py	docs/project_structure.md / .docx / .pdf

13. Правила: куда класть и что можно удалить

Куда класть новое:

- новый код пайплайна — в src/ отдельным модулем, описание в первом docstring;
- то, что запускается руками, — в scripts/ одной командой и одним абзацем в docstring;
- промежуточные таблицы и метрики — в data/processed/ с префиксом задачи (voltage_*, capacity_*, rul_*);
- графики — в reports/figures/ с нумерацией 01_...10_ либо именем задачи;
- документы для человека — в docs/ (markdown) и reports/ (отчёт), HTML собирается скриптом, а не правится руками;
- тест на каждое утверждение о данных — в tests/, на каждую проверку честности пайплайна — в scripts/check_*.py.

Что можно удалить без последствий для работы:

- __pycache__/ и .pytest_cache/ — кэши, собираются заново;
- папка _trash/ — удалённые ранее файлы, в работе не участвуют;
- файлы в reports/figures/, на которые отчёт не ссылается — их 15 (приложение Б);
- в data/processed/ — метрики и веса моделей с пометками в имени, которые не читает ни один скрипт пайплайна;
- в scripts/ — один скрипт, который строит сравнительный график по двум наборам метрик: в текущем пайплайне он не вызывается, а его график и метрики не подключены ни к одному документу.

> После любого удаления пересоберите документ: py -X utf8 scripts/build_structure_doc.py — перечень файлов в приложениях обновится сам.

Приложение А. Перечень файлов

В перечне — все файлы папок, которые участвуют в пайплайне, отчёте или документации. Файлы от экспериментов, не вошедших в отчёт, посчитаны в приложении Б, но не названы: они не нужны ни для повторения работы, ни для проверки её честности.

А.1. data/raw/ — исходные файлы

Файл	Размер
B0005.mat	15.2 МБ
B0006.mat	15.3 МБ
B0007.mat	15.3 МБ
B0018.mat	8.1 МБ
B0025.mat	2.6 МБ
B0026.mat	2.6 МБ
B0027.mat	2.7 МБ
B0028.mat	2.7 МБ
B0029.mat	2.7 МБ
B0030.mat	2.7 МБ
B0031.mat	2.7 МБ
B0032.mat	2.8 МБ
B0033.mat	10.5 МБ
B0034.mat	10.6 МБ
B0036.mat	10.9 МБ
B0038.mat	3.5 МБ
B0039.mat	3.6 МБ
B0040.mat	3.6 МБ
B0041.mat	3.6 МБ
B0042.mat	7.2 МБ
B0043.mat	7.1 МБ
B0044.mat	7.2 МБ
B0045.mat	3.4 МБ
B0046.mat	3.5 МБ
B0047.mat	3.5 МБ
B0048.mat	3.6 МБ
B0049.mat	2.1 МБ
B0050.mat	2.1 МБ
B0051.mat	2.1 МБ
B0052.mat	2.1 МБ
B0053.mat	3.2 МБ
B0054.mat	6.8 МБ
B0055.mat	6.9 МБ

Файл	Размер
B0056.mat	6.9 МБ

A.2. data/processed/ — результаты пайплайна

Файл	Размер
battery_capacity.parquet	289 КБ
battery_diagnostics.csv	2 КБ
battery_features.csv	1.7 МБ
battery_features.json	3.7 МБ
battery_features.parquet	741 КБ
battery_features.xlsx	995 КБ
battery_features_report.md	1012 Б
battery_rul.parquet	120 КБ
battery_voltage.parquet	369 КБ
clean_cycles.parquet	428 КБ
comparison_baselines.json	2 КБ
metrics_baselines_capacity.json	464 Б
metrics_baselines_rul.json	440 Б
metrics_baselines_voltage.json	466 Б
metrics_capacity.json	2 КБ
metrics_rul.json	1 КБ
metrics_voltage.json	2 КБ
model_capacity.keras	63 КБ
model_rul.keras	66 КБ
model_voltage.keras	65 КБ
raw_cycles.parquet	429 КБ
summary_baselines.json	2 КБ
summary_metrics.json	814 Б

> Ещё 11 файл(ов) в этой папке не показаны: они созданы экспериментами, которые в отчёт не вошли (приложение Б).

A.3. src/ — модули

Файл	Размер
__init__.py	0 Б
baselines.py	4 КБ
clean.py	3 КБ
config.py	5 КБ
export.py	60 КБ
features.py	4 КБ
html_report.py	5 КБ
infographics.py	26 КБ
main.py	9 КБ
model.py	8 КБ
parser.py	8 КБ
plots.py	6 КБ
rul.py	3 КБ
split.py	3 КБ
validate.py	4 КБ

A.4. scripts/ — скрипты

Файл	Размер
build_doc_html.py	6 КБ
build_html_report.py	668 Б
build_structure_doc.py	48 КБ
check_leakage.py	8 КБ
check_parser.py	4 КБ
diagnose_batteries.py	5 КБ
regen_figures.py	3 КБ
run_all.py	335 Б
run_baseline_chart.py	6 КБ
run_baselines.py	8 КБ
xml_to_html.py	6 КБ

> Ещё 1 файл(ов) в этой папке не показаны: они созданы экспериментами, которые в отчёт не вошли (приложение Б).

A.5. tests/ — тесты

Файл	Размер
test_features.py	3 КБ
test_parser.py	1 КБ
test_rul.py	2 КБ
test_split.py	2 КБ

A.6. reports/ — отчёт

Файл	Размер
figures/ · 33 файла	5.0 МБ
lab1_report.html	57 КБ
lab1_report.md	48 КБ
style.css	3 КБ
theory_manual.md	24 КБ

A.7. docs/ — документы

Файл	Размер
lab1_defense.html	24 КБ
lab1_defense.md	19 КБ
lab1_likbez.html	53 КБ
lab1_likbez.md	41 КБ
lab1_process.css	4 КБ
lab1_process.html	72 КБ
lab1_process.xml	52 КБ
lab1_process.xsd	16 КБ
lab1_process.xsl	20 КБ
project_structure.docx	47 КБ
project_structure.md	29 КБ
project_structure.pdf	310 КБ
style_likbez.css	6 КБ

A.8. artifacts/ — сводки

Файл	Размер
battery_stats.csv	2 КБ
no_eol_batteries.json	927 Б

Приложение Б. Что не используется

Эти файлы лежат на диске, но не читаются ни одним скриптом пайплайна и не подключены ни к одному документу. Удаление ничего не ломает: числа отчёта и графики строятся только по файлам из приложения А.

Где	Сколько	Что это	Можно удалить
data/processed/	11	метрики и веса моделей от экспериментов, которые в отчёт не вошли (276 КБ)	да
reports/figures/	8	графики тех же экспериментов	да
reports/figures/	7	вспомогательные графики базовых предсказателей и промежуточных запусков: отчёт на них не ссылается	да
scripts/	1	скрипт, строящий сравнительный график по двум наборам метрик: в пайплайне не вызывается	да
_trash/	—	удалённые ранее файлы и папки	да
__pycache__/, .pytest_cache/	—	кэши Python и pytest	да, собираются заново